

CTF & CTFL v1.1 – The Complete, Standalone Test Generation Ecosystem

From Static MCQs to Dynamic, Programmable Tests

A single guide for human authors, AI content generators, and software developers building the interpreter

What you will learn in this document: This guide starts from the very basics. You'll understand what CTF is, how to write a simple test, and then progressively build up to the full CTFL programming language with variables, loops, randomness, functions, and LaTeX math. No prior knowledge of any earlier version is required. The document is divided into two self-contained parts: *Part A – Writing Tests* (for anyone creating questions) and *Part B – Building the Parser* (for developers implementing the interpreter). Both parts are complete and can be used independently.

1. The Big Picture – What Are CTF and CTFL?

Imagine you have a study website that needs multiple-choice questions (MCQs). A single question contains:

- The question text (e.g., "What is 2+2?")
- Four options (a: 3, b: 4, c: 5, d: 6)
- The correct answer (b)
- An explanation (reason)
- Metadata: subject, chapter, topic, difficulty level, cognitive type, and stem category.

CTF (Custom Test Format) is a plain-text format to represent *one or more* such questions in a compact, machine-readable way. You can upload a CTF file to your platform, and it will appear as a ready-to-use test.

CTFL (Custom Test Format Language) extends CTF with programming capabilities: variables, loops, randomness, and reusable functions. With CTFL you can write a *script* that generates hundreds of distinct questions from a tiny template. The platform runs the script and produces the same kind of question objects that CTF would, so the GUI doesn't need to change at all.

Throughout this guide, we'll first learn CTF (the static part) and then add the dynamic features of CTFL. Even if you only ever use CTF, you can stop after the first part and be fully productive.

Part A – Writing Tests (For Human & AI Authors)

A.1 The Simplest Possible CTF Question

A CTF file is just a sequence of characters. Here's the smallest complete example of a CTF question:

```
Subj1:"Science"/Chap1:"Physics"/Topi1:"Gravity"/Type1:"K"/Diff1:"E"/Stem1:""/
( Q:"What is Earth's gravity?" A:"b" O:a)"8.9",b)"9.8",c)"10.0",d)"9.0" R:"Standard
value is 9.8 m/s2."; )
```

Even though it's written on one line, it contains all the information needed. Let's break it down piece by piece.

A.1.1 The Metadata Path

The first part before the `(` is the **path**. It's a list of **wrappers** separated by slashes. Each wrapper has a name, a number (the identifier), a colon, and a **double-quoted value**:

- `Subj1:"Science"` — the subject is "Science", identifier 1.
- `Chap1:"Physics"` — chapter "Physics".
- `Topi1:"Gravity"` — topic "Gravity".
- `Type1:"K"` — cognitive type: Knowledge (K).
- `Diff1:"E"` — difficulty: Easy (E).
- `Stem1:""` — stem type: empty (no specific category).

Important: The six wrappers must appear in this exact order: Subj, Chap, Topi, Type, Diff, Stem. The identifier (`1`) helps the parser distinguish different instances of the same wrapper type, but for beginners you can always use `1` .

A.1.2 The Question Container

After the path, a pair of parentheses `( ... )` holds the question data. Inside, the fields are:

- `Q:"question text"`
- `A:"correct letter"` (must be quoted)
- `O:a)"text", b)"text", c)"text", d)"text"` — options, each with a letter and closing parenthesis `)` , separated by commas.
- `R:"reason"`

You can put multiple questions inside the same container by separating them with semicolons. For example:

```
Subj1:"Math"/Chap1:"Arithmetic"/Topi1:"Addition"/Type1:"K"/Diff1:"E"/Stem1:""/
( Q:"2+2?" A:"b" O:a)"3",b)"4",c)"5",d)"6" R:"2+2=4"; Q:"3+5?" A:"c"
O:a)"7",b)"9",c)"8",d)"10" R:"3+5=8"; )
```

This single block generates two questions that share the same metadata.

✓ DO – CTF Essentials

- Always include all six wrappers in the path.
- Put all string values in double quotes. For **Type** and **Diff**, the values are still quoted (we'll relax this later in CTFL).
- Use the correct order: Subj, Chap, Topi, Type, Diff, Stem.
- Separate questions inside a container with **;**.

✗ DON'T – CTF Pitfalls

- Don't omit a wrapper — the parser expects all six.
- Don't use bare letters for **Type** or **Diff** in strict CTF; they must be quoted.
- Don't mix up the field order — the parser expects Q, A, O, R inside each question, though spaces are ignored.

A.2 Moving from CTF to CTFL – Adding Variables

CTFL is an extension of CTF. It allows you to **write a script** instead of a static list of questions. The simplest addition is **variables**.

You can define a variable with a dollar sign and use it inside any string. For example:

```
$x = 5;
$y = 7;
q {
  Q:"Add $x and $y"
  A:"c"
  O:a)"11", b)"13", c)"12", d)"10"
  R:"$x + $y = 12"
}
```

Here, **q { }** is a CTFL question block (an alternative to the parentheses). It uses the current metadata, which you set with a **meta** statement or a path block.

A.2.1 Setting Metadata in CTFL

You can still use the full path like before, but CTFL adds the **meta** statement for flexibility:

```
meta Subj1:"Math"/Chap1:"Algebra"/Topi1:"Equations"/Type1:"U"/Diff1:"M"/
Stem1:"calculation"/
```

After this line, all subsequent **q** blocks inherit that metadata. You can also change just one wrapper later:

```
meta Diff1:"H"/ // make the next questions hard
```

A **path block** creates a temporary scope:

```
Subj1:"Bio"/Chap1:"Genetics"/Topi1:"DNA"/Type1:"K"/Diff1:"E"/Stem1:""/ {
  q { Q:"What is DNA?" A:"a" O:a)"molecule",b)"cell",c)"atom",d)"organ" R:"DNA is a
molecule." }
}
```

Inside the braces, the path is active; after the braces, the old metadata returns.

✓ DO – CTFL Scripts

- Use `meta` to set the global metadata at the top.
- Use variables to make questions dynamic.
- Use `q { }` blocks for each question — it's the clearest style.

✗ DON'T – CTFL Scripts

- Don't forget to assign a value to a variable before using it.
- Don't use a variable name that starts with a number or contains special characters except underscores.

A.3 Expressions and Computation

CTFL supports mathematical expressions. You can use arithmetic operators (`+` `-` `*` `/` `%`), comparisons (`==` `!=` `<` `>` `<=` `>=`), logical operators (`&&` `||` `!`), and even a ternary conditional (`condition ? value1 : value2`).

Inside a double-quoted string, you can embed an expression with `$(...)` or just `$variable`.

Example:

```
$a = rand(1, 10);
$b = rand(1, 10);
$sum = $a + $b;
q {
  Q:"$a + $b = ?"
  A:"b"
  O:a)"$(sum-1)", b)"$sum", c)"$(sum+1)", d)"$(a*b)"
  R:"Add: $a + $b = $sum."
}
```

A.4 Randomness & Seeding

CTFL provides three random functions:

- `rand(min, max)` – random integer between min and max (inclusive).
- `pick(list)` – random element from a list.
- `shuffle(list)` – returns a new list with elements randomly ordered.

To ensure reproducibility (the same test every time you run the script), start with a seed:

```
@seed 12345;
```

If you don't set a seed, a default of 0 is used, so the output is still deterministic.

✓ DO – Randomness

- Always set a seed at the beginning of your script.
- Use `rand()` to generate variable numbers in questions.
- Use `shuffle()` to randomise option order — but remember to track which option is correct!

A.5 Loops and Conditionals

Loops let you create many questions from a single template:

- `@repeat 5 { ... }` – repeat the block exactly 5 times.
- `@for i in [1..10] { ... }` – iterate over a range or list.
- `@while (condition) { ... }` – repeat as long as condition is true (limited to 100,000 total iterations for safety).

Conditional execution:

```
@if ($level == 2) {
  meta Diff1:"H"
} @else {
  meta Diff1:"E"
}
```

✓ DO – Loops

- Use `@repeat` to generate a set of similar practice questions.
- Use `@for` with lists of words or data to create varied questions.

✗ DON'T – Loops

- Don't create infinite loops. The interpreter will stop after 100k total statements.
- Don't forget that `@while` still counts towards the global tick limit.

A.6 Functions – Reusable Templates

When you have a question pattern you want to reuse, wrap it in a function:

```
$add_drill(a, b) = {
  q {
    Q: "$a + $b = ?"
    A: "c"
    O: a) "$(a+b-1)", b) "$(a+b+1)", c) "$(a+b)", d) "$(a*b)"
    R: "Sum."
  }
};
```

Call it: `$add_drill(5, 7);`

Functions are completely isolated – they can't see or change variables outside them.

✓ DO – Functions

- Define functions near the top of your script for clarity.
- Use descriptive names like `$quadratic_solver`.
- Call functions with a semicolon after the closing parenthesis.

✗ DON'T – Functions

- Don't make a function call itself (recursion is not allowed).
- Don't assume a function can access global variables — pass everything as parameters.

A.7 LaTeX Math – Beautiful Equations

Inside any string (question text, options, reason) you can embed LaTeX math for perfect rendering. Use **safe delimiters** that do not clash with CTFL variables:

- Inline: `\(...\)` – write as `\(...\)` in CTFL strings (backslash doubled).
- Display: `\[...\]` – write as `\[...\]`.

Example:

```
q {
  Q:"Solve \((x^2 - 4 = 0)\)."
  A:"a"
  O:a)"\((x = 2, -2)\)", b)"\((x = 4, -4)\)", c)"\((x = 0)\)", d)"\((x = 1, -1)\)"
  R:"Factor: \((x-2)(x+2)=0)\)."
}
```

The interpreter passes the string with single backslashes to the frontend, which then uses KaTeX or MathJax to render the math.

✓ DO – Math

- Use `\(` and `\)` for inline math.
- Double the backslashes in CTFL strings.

✗ DON'T – Math

- Don't use `$.$.` for math — it will be confused with CTFL variables.
- Don't forget to escape backslashes — a single `\\` is the escape character.

A.8 Complete Mini-Script Example (CTFL)

```
@seed 42;
meta Subj1:"Math"/Chap1:"Arithmetic"/Topi1:"Addition"/Type1:U/Diff1:E/Stem1:"drill"/;

$add_question = {
  $a = rand(10, 99);
  $b = rand(10, 99);
  q {
    Q:"$a + $b = ?"
    A:"c"
    O:a)"$(a+b-1)", b)"$(a+b+1)", c)"$(a+b)", d)"$(a*b)"
    R:"Add the two numbers."
  }
};

@repeat 10 { $add_question(); }
```

This single script generates 10 unique addition problems. If you change the seed, you get a different set, but the same seed always gives the same set.

Part B – Building the Parser & Interpreter (For Developers)

This section is a complete reference for implementing a CTF/CTFL engine. It assumes you are familiar with building lexers and parsers.

B.1 Language Philosophy & Compatibility

CTFL v1.1 is a **superset of CTF**. Every valid CTF string must parse and execute correctly in the CTFL interpreter. To achieve this, the grammar includes explicit tolerance rules that forgive common punctuation habits.

Tolerance Rules (Mandatory)

1. **Trailing semicolons after `}`** are ignored (treated as empty statements).
2. **Inside a `q` block**, semicolons between the `Q`, `A`, `O`, `R` fields are optional and ignored.
3. **Type and Diff wrappers** accept bare identifiers `K`, `U`, `A`, `E`, `M`, `H` (without quotes). Any other value must be a quoted string. Quoted forms are still valid for backward compatibility.
4. **Lone semicolons (`;`)** are valid empty statements anywhere.
5. **Legacy parentheses (`...`)** for question containers are fully supported alongside `q { }`.

B.2 Lexer Specification

The lexer must produce tokens for:

- Identifiers: `[a-zA-Z_][a-zA-Z0-9_]*`
- Keywords: `meta`, `q`, `true`, `false`, `null`, `@if`, `@else`, `@for`, `@in`, `@while`, `@repeat`, `@import`, `@seed`
- Numbers: integers only (floats not needed, but may be parsed for future use).
- Double-quoted strings with escapes (`"`, `\`, `\$`) and **interpolation**. When encountering `$identifier` or `$(` inside a string, the lexer must emit special tokens that allow the parser to embed an expression tree. A common approach is to break the string into segments (text, interpolation) and let the parser combine them.
- Comments: `//` to end of line, discarded.
- Operators and delimiters as per the grammar below.

B.3 Complete Grammar (EBNF) with Tolerances

This grammar reflects all the tolerance rules. It is the canonical specification.


```

program      = { statement } EOF

statement    = meta_stmt
              | path_block
              | question_block
              | func_def
              | assign_stmt ";"
              | expr_stmt ";"
              | control_stmt
              | import_stmt ";"
              | seed_stmt ";"
              | ";" (* empty statement *)

meta_stmt    = "meta" path ";"
path         = segment { "/" segment } [ "/" ]
segment      = wrapper ":" value
wrapper      = ("Subj" | "Chap" | "Topi" | "Type" | "Diff" | "Stem") INTEGER
value        = STRING
              | BARE_CODE (* only when wrapper is Type or Diff *)
BARE_CODE    = "K" | "U" | "A" | "E" | "M" | "H"

path_block   = path "{" { statement } "}" [ ";" ]

question_block = "q" "{" [ question_body ] "}" [ ";" ]
question_body  = question_field { [ ";" ] question_field }
question_field = "Q" ":" expression
              | "A" ":" expression
              | "O" ":" option_list
              | "R" ":" expression
option_list    = option { "," option }
option         = expression ")" expression

func_def      = "$" IDENTIFIER "(" [ param_list ] ")" "=" "{" { statement }
              "}" [ ";" ]
param_list    = IDENTIFIER { "," IDENTIFIER }

assign_stmt   = "$" IDENTIFIER "=" expression
expr_stmt     = expression

control_stmt  = if_stmt | for_stmt | while_stmt | repeat_stmt
if_stmt       = "@if" "(" expression ")" block [ "@else" block ]
for_stmt      = "@for" IDENTIFIER "in" expression block
while_stmt    = "@while" "(" expression ")" block
repeat_stmt   = "@repeat" expression block
block         = "{" { statement } "}" [ ";" ]

```

```

expression      = assignment_expr
assignment_expr = ternary_expr [ "=" assignment_expr ] (* right associative *)
ternary_expr    = logical_or [ "?" expression ":" ternary_expr ]
logical_or      = logical_and { "||" logical_and }
logical_and     = equality { "&&" equality }
equality        = comparison [ ("==" | "!=") comparison ]
comparison      = additive [ ("<" | ">" | "<=" | ">=") additive ]
additive         = multiplicative { "+" | "-" multiplicative }
multiplicative  = unary { ("*" | "/" | "%") unary }
unary           = [ "-" | "!" ] primary
primary         = literal
                  | IDENTIFIER
                  | "$" IDENTIFIER
                  | func_call
                  | "(" expression ")"
                  | list_expr
                  | range_expr
literal         = INTEGER | STRING | "true" | "false" | "null"
func_call       = "$" IDENTIFIER "(" [ expression { "," expression } ] ")"
list_expr       = "[" [ expression { "," expression } ] "]"
range_expr      = "[" expression ".." expression "]"

IDENTIFIER      = letter { letter | digit | "_" }
INTEGER         = digit { digit }
STRING          = "'" { char_no_quote | "'" | '\\' | '\\$' | interpolation } "'"
                  (* interpolation: "$(" expression ")" or "$" IDENTIFIER *)
COMMENT         = "//" { any_char_except_newline } newline

```

B.4 Interpreter Execution Model

1. **Parse** the source into an AST.
2. **Resolve imports** at parse time (or before execution) – detect circular imports.
3. **Execute** statements sequentially. Maintain:
 - Metadata context (a map from wrapper -> (id, value)).
 - Variable scope stack (global, block, function scopes).
 - Random number generator seeded by `@seed`.
 - Output list of question objects.
 - Tick counter (increment per statement/expression evaluation).
4. **Loop safety:** If tick counter exceeds 100,000, halt with error.
5. **Function execution:** Create a new scope with only parameters. No access to outer variables. Check for recursion (direct or indirect) before calling.
6. **Output:** Serialize the list of question objects to JSON (as shown in §B.5).

B.5 JSON Output Schema

Each question object:

```
{
  "subject": "Mathematics",
  "chapter": "Algebra",
  "topic": "Quadratic Equations",
  "type": "U",
  "difficulty": "M",
  "stemType": "problem",
  "question": "Solve  $(x^2 - 4 = 0)$ .",
  "options": {
    "a": " $(x = 2, -2)$ ",
    "b": " $(x = 4, -4)$ ",
    "c": " $(x = 0)$ ",
    "d": " $(x = 1, -1)$ "
  },
  "correctAnswer": "a",
  "reason": "Factor:  $(x-2)(x+2)=0$ ."
}
```

LaTeX delimiters remain as text; the frontend renders them.

B.6 Error Handling & Safety Summary

- Unknown tokens, mismatched quotes, missing fields → syntax error with line number.
- Undefined variable, type mismatch, division by zero → runtime error.
- Function call depth or recursion detected → error.
- Import file not found, circular import → error.
- Tick counter exceeded → halted.

Appendix A – Quick Reference: All CTFL Tokens & Delimiters

Token	Meaning
Subj	Subject wrapper
Chap	Chapter wrapper
Topi	Topic wrapper
Type	Cognitive type (K/U/A) – can be bare or quoted
Diff	Difficulty (E/M/H) – can be bare or quoted

Stem	Stem type
meta	Update metadata
q { }	Question block
Q / A / O / R	Question fields
@seed	Set random seed
@repeat	Fixed repetition loop
@for ... in ...	Iterate over list/range
@while	Conditional loop
@if / @else	Conditional
@import "file"	Include another script
\$name	Variable
\$(expr)	Expression interpolation
rand(), pick(), shuffle(), str(), len()	Built-in functions
//	Line comment
;	Statement terminator (optional in many places)
"	String delimiter
\(... \)	Inline LaTeX math (doubled backslash in source)
\[... \]	Display LaTeX math

Appendix B – The Ultimate Do's & Don'ts for Test Authors

✓ ALWAYS DO

- Start with `@seed` and a full `meta` path.
- Use `q { }` for each question.
- Quote the correct answer letter (`"c"`).
- Use `\(` for math and double backslashes.
- Put semicolons after function calls and variable assignments.
- Keep functions pure – pass everything as parameters.

✗ NEVER DO

- Don't use unquoted text for `Subj` , `Chap` , `Topi` , `Stem` — only `Type` / `Diff` may be bare.
- Don't forget to close strings and braces.
- Don't use `$. . $.` for LaTeX inside strings.
- Don't rely on global variables inside functions.
- Don't create recursive functions.
- Don't exceed the 100k tick limit with endless loops.

Appendix C – Tips for AI Generators

If you are an AI producing CTFL scripts, follow these rules to avoid common errors:

- **Always output complete, runnable scripts.** Start with `@seed` and a `meta` line.
- **Use `\( . . . \)` for all math.** Double the backslashes.
- **Enclose all text in double quotes.** Only bare `Type` and `Diff` values are allowed.
- **Separate function calls with semicolons.**
- **Avoid infinite loops.** Use `@repeat` or `@for` with fixed bounds.

Appendix D – From Zero to Test: A Learning Path

1. Start by writing a single CTF question (path + parentheses).
2. Convert it to a `q { }` block with a `meta` statement.
3. Introduce a variable and use it in the question text.
4. Add `rand()` to generate different numbers each run.
5. Wrap the pattern in a function and call it several times.
6. Use `@repeat` to create a full drill.

7. Include LaTeX math for science and math subjects.

By following these steps, anyone can go from knowing nothing to creating complex, dynamic test banks.

End of CTF & CTFL Standalone Guide – Happy Testing!